Skill - Week6

Session 11:

1. To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency.

```c
#include <stdio.h>
#include <string.h>
#define MAX_HISTORY 5
#define MAX_COMMAND 100
char history[MAX_HISTORY][MAX_COMMAND];
int count = 0;
int next = 0;
/* Store a command */
void store_command(char *cmd) {
    strncpy(history[next], cmd, MAX_COMMAND - 1);
    history[next][MAX_COMMAND - 1] = '\0';
    next = (next + 1) % MAX_HISTORY;
    if (count < MAX_HISTORY)
        count++;
}
/* Retrieve a command */
void retrieve_command(int index) {
    if (index < 1 || index > count) {
        printf("Invalid history index\n");
        return;
    }
    int start = (next - count + MAX_HISTORY)
                % MAX_HISTORY;
    int position = (start + index - 1) % MAX_HISTORY;
    printf("Command %d: %s", index, history[position]);
}
/* Display history */
void display_history() {
    if (count == 0) {
        printf("History is empty\n");
        return;
    }
    int start = (next - count + MAX_HISTORY)
                % MAX_HISTORY;
    for (int i = 0; i < count; i++) {
       int position = (start + i) % MAX_HISTORY;
        printf("%d: %s", i + 1, history[position]);
```

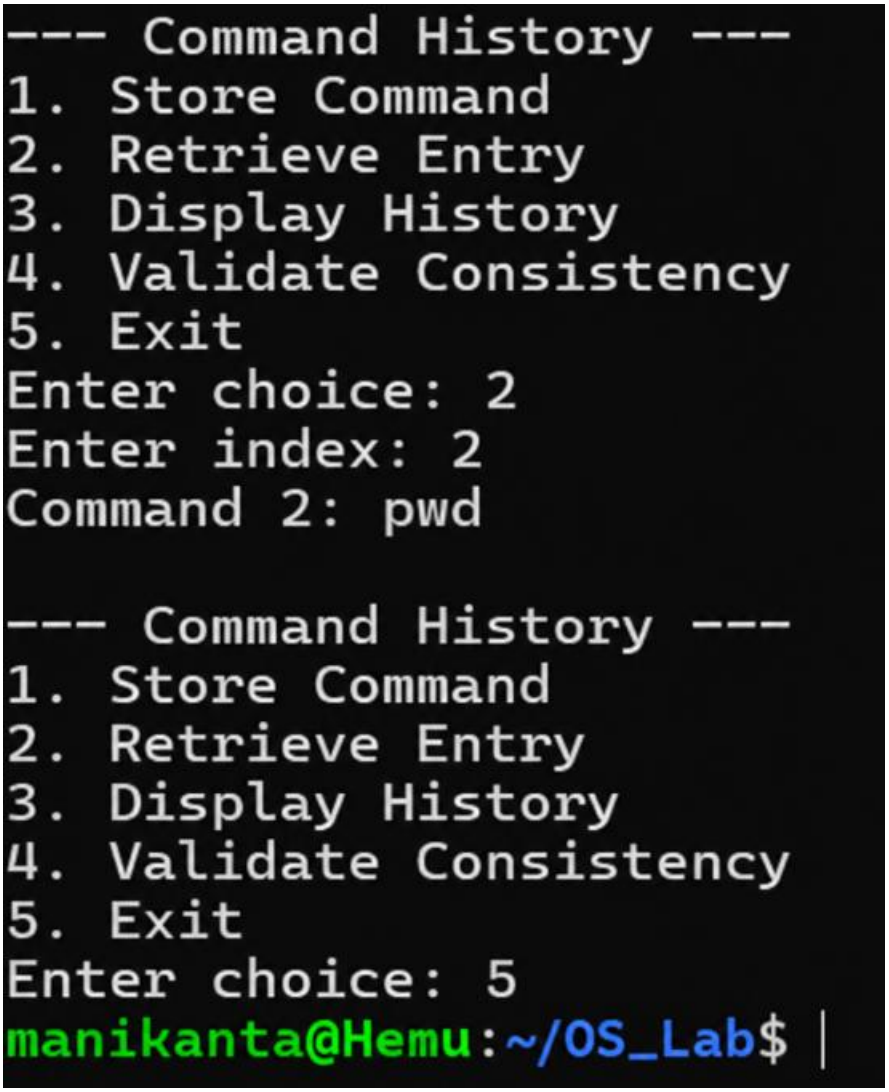
```
manikanta@Hemu:~$ cd OS_Lab
manikanta@Hemu:~/OS_Lab$ nano command_history.c
manikanta@Hemu:~/OS_Lab$ gcc command_history.c -o command_history
manikanta@Hemu:~/OS_Lab$ ./command_history

--- Command History ---
1. Store Command
2. Retrieve Entry
3. Display History
4. Validate Consistency
5. Exit
Enter choice: 1
Enter command: ls
Command stored successfully

--- Command History ---
1. Store Command
2. Retrieve Entry
3. Display History
4. Validate Consistency
5. Exit
Enter choice: 1
Enter command: pwd
Command stored successfully

--- Command History ---
1. Store Command
2. Retrieve Entry
3. Display History
4. Validate Consistency
5. Exit
Enter choice: 3
1: ls
2: pwd

--- Command History ---
1. Store Command
2. Retrieve Entry
3. Display History
4. Validate Consistency
5. Exit
```



Report: The program stores commands in a fixed-size history buffer. A circular buffer handles capacity limits by overwriting the oldest entries. Users can retrieve specific commands, display the complete history, and validate the buffer's consistency.

2. To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#define MAX_COMMANDS 5
#define MAX_ARGS 10
#define MAX_LENGTH 100
typedef struct {
    char command[MAX_LENGTH];
} Command;
typedef struct {
    Command commands[MAX_COMMANDS];
    int count;
} Pipeline;
/* Initialize pipeline */
void init_pipeline(Pipeline *p) {
    p->count = 0;
}
/* Store command */
void store_command(Pipeline *p, char *cmd) {
    if (p->count >= MAX_COMMANDS) {
        printf("Error: Pipeline capacity exceeded\n");
        return;
    }
    strcpy(p->commands[p->count].command, cmd);
    p->count++;
    printf("Command stored successfully\n");
}
/* Display pipeline */
void display_pipeline(Pipeline *p) {
    printf("\nPipeline Layout:\n");
    for (int i = 0; i < p->count; i++) {
        printf("%d: %s\n", i + 1,
               p->commands[i].command);
    }
}
```

^G Help ^O Write Out ^F Where Is ^K Cut

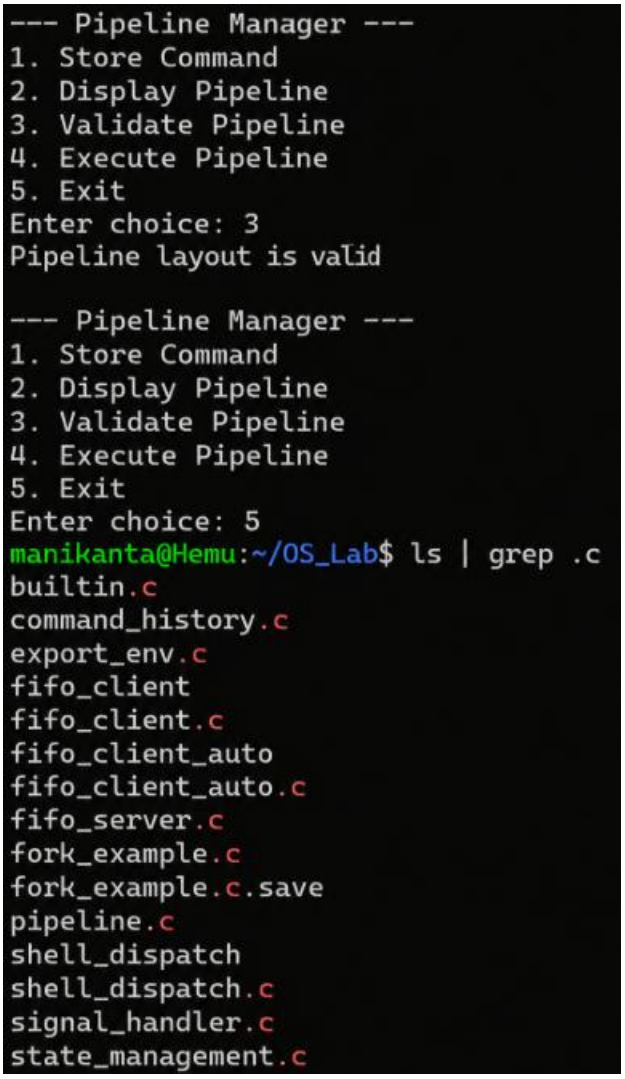
```
manikanta@Hemu:~/OS_Lab$ nano pipeline.c
manikanta@Hemu:~/OS_Lab$ gcc pipeline.c -o pipeline
manikanta@Hemu:~/OS_Lab$ ./pipeline

--- Pipeline Manager ---
1. Store Command
2. Display Pipeline
3. Validate Pipeline
4. Execute Pipeline
5. Exit
Enter choice: 1
Enter command: grep
Command stored successfully

--- Pipeline Manager ---
1. Store Command
2. Display Pipeline
3. Validate Pipeline
4. Execute Pipeline
5. Exit
Enter choice: 1
Enter command: ls
Command stored successfully

--- Pipeline Manager ---
1. Store Command
2. Display Pipeline
3. Validate Pipeline
4. Execute Pipeline
5. Exit
Enter choice: 2

Pipeline Layout:
1: grep
2: ls
```

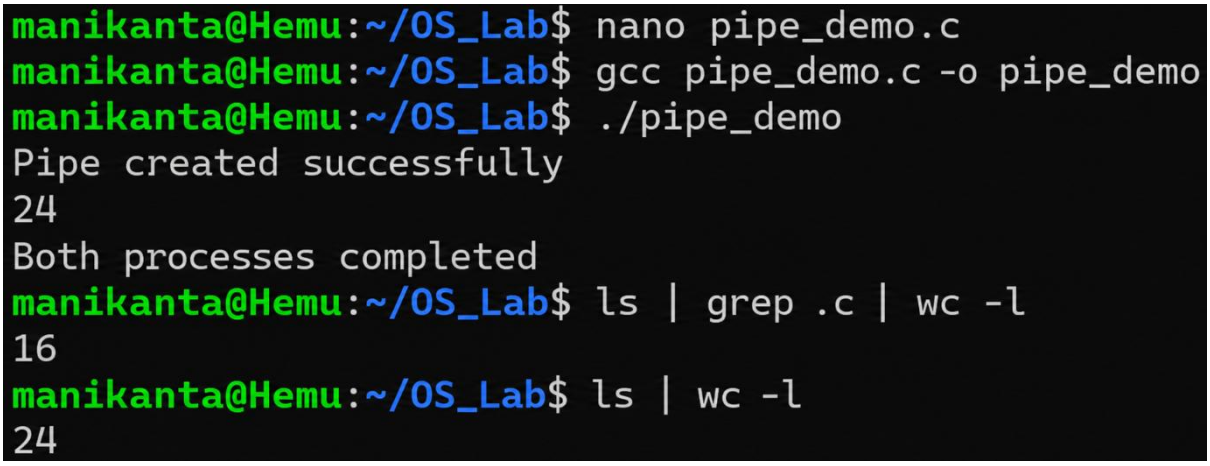


Report: The program creates a pipeline structure to store commands in order. It validates the number of commands and uses Linux system calls such as pipe(), fork(), dup2(), and waitpid() to connect processes and execute commands.

Session 12:

1. To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>
int main() {
    int fd[2];
    pid_t pid1, pid2;
    /* Create pipe */
    if (pipe(fd) == -1) {
        perror("pipe");
        exit(1);
    }
    printf("Pipe created successfully\n");
    /* First child process */
    pid1 = fork();
    if (pid1 == -1) {
        perror("fork");
        exit(1);
    }
    if (pid1 == 0) {
        /* Redirect stdout to pipe */
        dup2(fd[1], STDOUT_FILENO);
        /* Close unused descriptors */
        close(fd[0]);
        close(fd[1]);
        /* Execute first command */
        execlp("ls", "ls", NULL);
        perror("execlp");
        exit(1);
    }
    /* Second child process */
    pid2 = fork();
    if (pid2 == -1) {
        perror("fork");
        exit(1);
    }
```



Result: The pipe was created successfully. The output of ls was redirected to wc -l, and both child processes completed successfully. Data flow was verified using Linux pipeline commands.

2. To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#define MAX_COMMANDS 5
int main() {
    /* Commands in execution order */
    char *commands[MAX_COMMANDS][4] = {
        {"ls", NULL, NULL, NULL},
        {"grep", ".c", NULL, NULL},
        {"wc", "-l", NULL, NULL}
    };
    int n = 3;
    /* Descriptor array */
    int pipes[MAX_COMMANDS - 1][2];
    pid_t pids[MAX_COMMANDS];
    /* Create multiple pipes */
    for (int i = 0; i < n - 1; i++) {
        if (pipe(pipes[i]) == -1) {
            perror("pipe");
            exit(1);
        }
        printf("Pipe %d created\n", i + 1);
    }
    /* Launch multiple processes */
    for (int i = 0; i < n; i++) {
        pids[i] = fork();
        if (pids[i] == -1) {
            perror("fork");
            exit(1);
        }
        if (pids[i] == 0) {
            /* Redirect input from previous pipe */
            if (i > 0) {
                dup2(pipes[i - 1][0], STDIN_FILENO);
            }
            /* Redirect output to next pipe */
```

```
manikanta@Hemu:~/OS_Lab$ nano multi_pipe.c
manikanta@Hemu:~/OS_Lab$ gcc multi_pipe.c -o multi_pipe
manikanta@Hemu:~/OS_Lab$ ./multi_pipe
Pipe 1 created
Pipe 2 created

All processes completed
All pipe descriptors closed
```

```
manikanta@Hemu:~/OS_Lab$ ls | grep .c | wc -l
18
manikanta@Hemu:~/OS_Lab$ ls | grep .c | sort | wc -l
18
manikanta@Hemu:~/OS_Lab$ ls | grep .c | sort | head -n 5 | wc -l
5
```

Report: The program supports multiple pipes by using a two-dimensional descriptor array. It launches one child process for each command, redirects input and output using dup2(), executes commands using execvp(), closes unused descriptors, and synchronizes all processes using waitpid(). Multiple pipeline configurations are tested to verify data flow and resource cleanup.